

New test case prioritization strategies for regression testing of web applications

Deepak Garg · Amitava Datta · Tim French

Received: 15 September 2012/Revised: 6 October 2012/Published online: 1 November 2012

© The Society for Reliability Engineering, Quality and Operations Management (SREQOM), India and The Division of Operation and Maintenance, Lulea University of Technology, Sweden 2012

Abstract A test case prioritization technique reschedules test cases for regression testing in an order to achieve specific goals like early fault detection. Currently, black box testing of web applications requires manual prioritization of test cases which is very time consuming. **We propose a new two level prioritization approach to prioritize test cases for web applications as a whole.** Our approach selects modified functionalities in a web application and automatically executes test cases on the basis of the impact of modified functionalities. **We suggest several new prioritization strategies for web applications and examine whether these prioritization strategies improve the rate of fault detection for web applications.** We propose a new **test suite prioritization model for web applications that detects modified functionalities, selects test cases related** to those functionalities and reschedules them to detect faults early in test suite execution.

Keywords Regression testing · Test case prioritization · Web applications · Prioritization strategies

1 Introduction

Over the last two decades web-based software systems or web applications have grown at a rapid rate. It is very important to

ensure the quality and reliability of these applications as many are used by a large number of users running large businesses. These applications are very complex and require frequent changes in their functionalities for supporting diverse user requirements (Mansour and Houri 2006). Therefore software testing of web applications is particularly challenging and failure may be very costly. Many web applications must have extremely high availability.

Web applications require more testing and maintenance than any other software systems (Elbaum et al. 2003). The testing of these applications differs considerably from traditional software systems due to many technologies used for developing different modules. Web applications typically use a complex and multi-tiered, heterogeneous architecture including web servers, application servers, database servers and client interpreters and testing must handle all these components. The main aim of testing of these applications is to detect faults in the required services/functionalities to enable them to function as per specifications (Lucca and Fasolino 2006). The major problem in testing is at unit testing level where a unique definition of scope of a unit test cannot be provided due to the heterogeneous architecture used in these applications (Di Lucca et al. 2002). As web applications have rapid development and testing cycles, many of the testing methods proposed in the literature cannot be used in a rapid testing environment as they are very time consuming.

As customer functionality requirements increase, web applications grow in size. The functional test cases are designed to test the functionalities as per the user specifications. For every new version of a web application released, regression testing is required to test the compatibility of the new features with the previous already tested functionalities. To perform regression testing, new test

D. Garg (✉) · A. Datta · T. French
School of Computer Science and Software Engineering,
The University of Western Australia, Perth, Australia
e-mail: deepak@csse.uwa.edu.au

A. Datta
e-mail: amitava.datta@uwa.edu.au

T. French
e-mail: tim@csse.uwa.edu.au

cases are generated. As the test cases grow in number due to the growth of applications, it becomes very difficult to execute all the test cases within a specified amount of time. The cost of re-running all test cases may be expensive and not always useful as sometimes only selected functionalities need to be tested. So, a number of test cases need to be re-executed using the various prioritization techniques (Baresi and Pezè 2006). Test cases are prioritized during the system testing in an order to discover the likely vulnerable parts of the code early so that developers have more time to identify and debug the faults.

We argue that it is important to detect faults (Guo and Sampath 2008) in functionalities of a web application early. Due to portability issues across different browsers and many other unique characteristics of web applications, there are possibilities of many appearance faults, form faults and other functionality issues. A logical fault in the program code relates to business logic and control flow e.g., if the user inputs the same string for the *password* and *confirm password* fields, still the application displays the error message *Password and Confirm Password fields don't match*. Form faults in the program code modifies and displays name-value pairs in forms. Appearance faults controls the way in which a web page is displayed.

Hence, there is a need for black box testing of web applications. The testing and prioritization techniques available in the literature mostly detect faults at the code level. Code level faults are different from functionality faults. If we apply these code level techniques, it may be possible that the faults related to functionality issues in web applications will be detected at the end of a test suite execution or the faults may not be detected if we select a subset of cases within a test suite.

To resolve the issue of higher test execution time, strategies have been proposed for prioritizing C (Elbaum et al. 2000; 2002) and Java programs (Do et al. 2006; Harrold et al. 2001; Jeffrey and Gupta 2006). For web applications, Sampath et al. (2008) suggested prioritization techniques using user-session based test cases.

In this paper, we expand upon the prioritization of functional test cases for testing of web applications. We suggest a two-level prioritization approach for prioritization of functional test cases for web applications based on the work of Rothermel et al. (2000). The main contributions of this paper are: (1) Detection of the new/modified functional modules in web applications. (2) Strategies to prioritize functional test suite for web applications. (3) An approach to prioritize the test cases related to new/modified functionalities. (4) New approach for testers to detect more faults earlier in test suite execution.

Section 2 presents the background study related to prioritization techniques. Section 3 presents the work related to generation of functional test cases from UML Activity

diagrams and generation of functionality graph. Section 4 presents the prioritization framework. Section 5 presents the prioritization heuristics. Section 6 describes our prioritization strategies. Section 7 describes the experimental evaluation. The results are presented in Sect. 8 and the conclusions are presented in Sect. 9.

2 Background

While there are many prioritization techniques available for software testing, very few are available for web application testing. Alberto et al. (Gonzalez-Sanchez et al. 2011) suggested a diagnostic prioritization technique that maximizes diagnostic information for every test case and minimizes loss of diagnostic quality in the prioritized test suite. Using this technique, tests are selected dynamically on the basis of the actual pass/fail results of earlier executed tests, resulting in a higher diagnostic performance per test.

Srikanth et al. (2005) suggested another cost effective test case prioritization technique that considers defect severity. This technique improves detection of severe faults during the testing of new code and regression testing of existing code known as prioritization of requirements for test (PORT). This technique prioritizes black box tests at the system level when information between requirements, test case, and test failures is maintained by the software development team.

Srivastava and Thiagarajan (2002) presented a technique for prioritization of test cases on the basis of coverage of statements, using change information and feedback approach. This technique is used for regression testing during development and is different from other techniques because of the calculation of flow graphs and coverage using binaries. They used this form of binary testing to identify vulnerabilities in pre-release software, which is usually shipped in binary format (Li et al. 2009).

Sampath et al. (2008) suggested a test suite prioritization technique for web applications using test lengths, frequency of appearance of request sequences, systematic coverage of parameter values and their interactions. They considered the frequency of user requests and interaction of parameter values in the requests. Their test cases were designed according to user sessions based on a series of HTTP requests consisting of base requests and name-value pairs. A base request is a HTTP request to access both static and dynamic content in web pages. They used the entire test suite for execution but they ordered the test cases on the basis of user session requests to detect the faults early in test suite execution.

Elbaum et al. (2004) also proposed a cost effective prioritization technique that uses a test suite to improve fault detection rates. They applied different prioritization techniques to different programs for early detection of

faults. Their method is based on total function coverage prioritization, which orders test cases according to the number of functions they cover.

Rothermel et al. defined the problem of test suite prioritization in (Rothermel et al. 2001). Rothermel et al. (2000; 2001) suggested the prioritization of test cases when the software is written in one single language. Their technique constructs control graphs and uses those graphs to select the test cases related to modified version of that software.

None of these papers suggested the approach for functional testing of web applications that can be fully automated. It is possible to automate the approach proposed by Rothermel et al. when the software is written in a single language. This can be done through a static analysis of the entire software and identifying the function modules and their connectivity in terms of the calling sequences. However, when the software consists of different modules written in different languages, it is not possible to perform static analysis of the entire program as static analysis tools are based on the grammar of a particular language. It may be possible to manually construct a control flow graph by checking the connectivity between different modules, but this process is extremely time consuming.

Moreover, web applications are prone to rapid changes and such connectivities need to be established manually every time a module is changed. This is not possible when rapid turn around in testing is required. Hence, it is difficult to base control flow graphs for web applications on the underlying code. The proposed two-level approach generalizes the approach proposed by Rothermel et al. (2000; 2001) at a functional level. Moreover, this approach also generalizes the approach proposed by Sampath et al. (Sampath et al. 2008). Since all user sessions are sub-graphs of the FDG of a web application, our approach will be able to prioritize test cases for all possible user sessions.

3 Related work

Web applications are composed of a three-tier web architecture; a client, a server and a data store (Mansour and Houri 2006). The end-users are connected through the web application client interface to the internet. The user submits a request to the web server through the client. The web server then requests the application objects and the application objects synchronize with the database server. The application server then synchronizes with the web application for processing of the transaction.

3.1 Generation of test cases

Törsel et al. (2011) and Tung et al. (2010) suggested automatic generation of test cases for web applications.

They described the automated approach to generate the test cases for functional testing.

3.2 Functionality and functional modules

Functionality refers to user actions such as keyboard and mouse events required to navigate through web applications (Di Lucca 2003; Sampath et al. 2007). UML sequence diagrams provide information about functionality and the interaction among different objects in a web application (Cartaxo et al. 2007). Functionality is defined in the specification documents and is provided by the user interface (Heinecke et al. 2010). A functional module is a collection of different functions (at the code level) to enable the functionality to behave according to the specifications.

Any modification in the functionality of a web application affects other working functionalities. To test for faults that arise in the already tested functionalities due to changes in other functionalities, there is a need to perform regression testing. Regression testing is helpful in performing the compatibility analysis of new features with the previous ones and can be performed for any kind of program including web based applications or system applications.

3.3 Functionality dependency graph (FDG)

A (FDG) (Garg and Datta 2012; Samuel et al. 2005; Zimmermann and Nagappan 2007) is a directed graph that is used to describe the relationship between functionalities in web applications. We manually extract the FDG of the *Online Bookstore*¹ application by navigating the links on the web pages of a web application as shown in Fig. 1. The link is dependent upon the another link for the functionality to behave as per its specifications. We assume that the link that is connected with the source-code of a web application and is performing some specific action is considered as one functional module. There are some user-generated links on the web pages, these links are not considered as a part of FDG.

For an FDG $G = \{V, E\}$, the node set V is the set of functionalities in a web application and E is the set of directed edges that represents the dependency relationship among the functionalities. A directed edge from $m \in V$ to $n \in V$ indicates the functional dependency of the module n on module m . The test suite is called *bookstore* and contains various test sets. These test sets refer to the functional modules in the FDG. The FDG provides us with a calling relationship among the functionalities in the Online Bookstore application.

¹ available freely at <http://www.gotocode.com>.

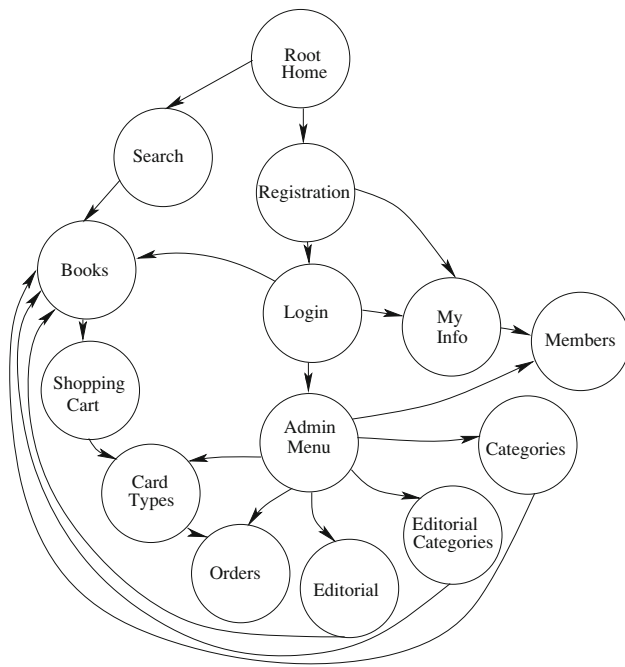


Fig. 1 The functionality dependency graph (FDG) for the *Online BookStore* application

Another possible approach of constructing the FDG is from the UML sequence diagram of the web application. This can be done by identifying the functionalities in a web application from the UML Sequence diagram of that web application. This can be used to capture various functional requirements and interactions between these requirements using UML. But depending upon the technologies, other methods of generation could be applied. Our prioritization method is a general method that may integrate many of these technologies.

This diagram provides information about the sequence of actions performed in an application. Each action is dependent on the previous action. Each functional module in FDG consists of functional sub-modules at the code level that enables the functionality to function as per its functional specifications.

4 Prioritization framework

In order to reduce the software testing cost and to improve the effectiveness of regression testing, various approaches like test case prioritization, test selection and test suite minimization have been proposed. We use a functionality dependency analysis approach for test case prioritization of web applications.

To perform regression testing, we propose a two level approach. We use a FDG that captures the functional dependencies among the functional modules in a web

application. We use an *inter-procedural control graph* (ICG) for each of the functional modules in the FDG. We construct ICGs within each functional module of the FDG using the technique proposed by Rothermel et al. (2000, 2001).

An example FDG in Fig. 1 shows functional modules in a web application. Each functional module in an FDG contains functional sub-modules. We have generated the ICG for each functional module in the FDG. We have sliced the entire test suite into different test sets related to the functionality. Each functional module in the FDG is tied to a test set and the sub-modules in the corresponding ICG are tied to the test cases contained in the test set. The FDG is used to prioritize the test sets and the ICG within a functional module of the FDG is used to prioritize the test cases within that test set.

Suppose an application A has m functionalities $f_1, f_2, \dots, f_m$. Hence the FDG for this application has m nodes and these functionalities have m corresponding test sets denoted by $TA = \{s_1, s_2, \dots, s_m\}$. We assume for simplicity that each functionality has n sub-modules at the code level, though the number of sub-modules in a functional module will in general vary. Hence each test set consists of n test cases, each test case covers a functional sub-module (positive and negative) in the corresponding ICG and the set of test cases corresponding to the test set $s_i, 1 \leq i \leq m$, are denoted by $TS_i = \{t_{i1}, t_{i2}, \dots, t_{in}\}$. We identify the test case $t_{ij}, 1 \leq j \leq n$, with the j -th functional sub-module for the ICG of $f_i \in FDG$.

In the next section, we will discuss the FDG with modified functionality i.e., when a functional module or a node in FDG has been modified. We say that a functional module f_i invokes a functional module f_j (denoted by a directed arrow from node f_i to f_j) if functionality f_j may follow functionality f_i during a user session. However, functionality f_j cannot be accessed by a user without first accessing functionality f_i . The FDG captures all possible invocation of functionalities by a user, however, a single user session may only access a subgraph of the FDG.

5 Prioritization heuristics

A web application consists of many functional modules. We identify the new/modified functionalities and separate them as new/modified functionalities in the FDG (Fig. 2). We first discuss the prioritization of the test sets for the FDG. We will next discuss the prioritization of the test cases for the ICG within each module of the FDG.

5.1 Prioritization of test sets for the FDG

Our technique for the prioritization of the test sets for the FDG consists of the following two steps:

- Identification of the nodes of the FDG that are affected by the modifications. Here we identify the nodes (or modules) of the FDG that need to be tested in the form of a subgraph of the FDG. This is the selection step.
- Prioritization of the test sets related to the nodes of the selected subgraph of the FDG. This is the prioritization step.

We assume that the specifications of a node $f \in FDG$ have changed. f is connected to the other nodes in the FDG in two different ways:

- f invokes other nodes in the FDG.
- f is invoked by other nodes in the FDG.

We make two assumptions on the faults in the nodes of the FDG:

- New/modified nodes may introduce new faults in the web application.
- Nodes that are invoked by a new/modified node and that have not changed after the previous testing cycle are assumed to be free of faults.

We note that both of these assumptions are realistic and reasonable, as the chance of introduction of new faults is the highest in new or modified nodes of the FDG. Also, nodes that have not changed, but are invoked by new or modified nodes remain fault free after the previous cycle of testing. For example, the node “Admin Menu” in Fig. 2 is invoked by the modified node “Login”. We assume that “Admin Menu” will be invoked correctly from “Login” even when the specification of “Login” has changed. Moreover,

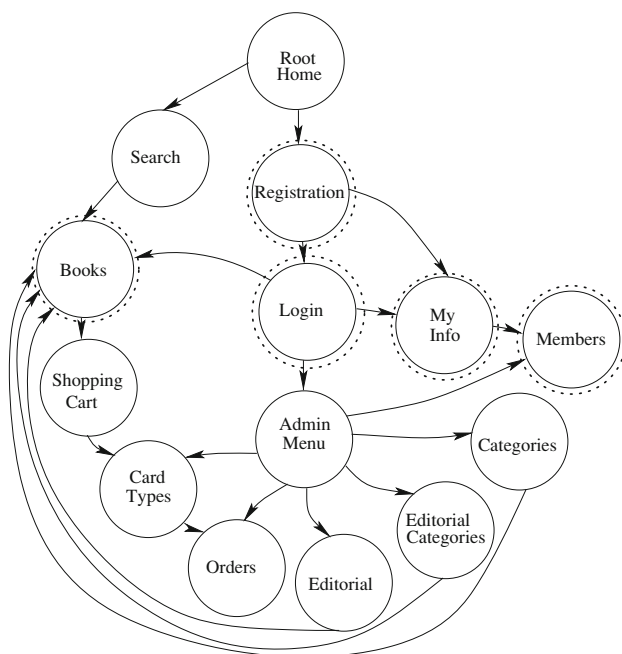


Fig. 2 The functionality graph with modified functionalities

“Admin Menu” was fault-free after the previous testing cycle and according to our assumption, it remains fault-free for the purpose of the current testing cycle. Hence, we assign highest priorities to the test sets for the new/modified nodes and least priorities for the test sets that are invoked by the new/modified nodes. The other nodes are assigned intermediate priorities as discussed below.

We say a node $f_i \in FDG$ directly invokes a node $f_j \in FDG$ if there is a directed edge from f_i to f_j . Similarly, a node $f_i \in FDG$ indirectly invokes a node $f_j \in FDG$ if there is a directed path from f_i to f_j . When a node $f_i \in FDG$ is new or modified, the nodes that directly or indirectly invoke f_i may or may not be affected for the following reasons:

- If f_i returns a different functionality, then it will affect the nodes that invoke f_i .
- If the source code of f_i is changed in terms of computation or functionality changes, but f_i is still providing the same functionality to the nodes that are invoking it, then it may or may not affect the invoking nodes.

We illustrate our proposed technique using a FDG (Fig. 2). We assume that the functional modules with dotted circles have been modified.

We assign priorities to the nodes of the FDG in the following way:

1. A newly introduced node in the FDG is given the highest priority. If there are multiple newly introduced nodes, they are assigned the highest priorities in an arbitrary order.
2. The modified nodes in the FDG are given the next lower priorities. The details of these priority assignments vary depending on the prioritization scheme used as we explain in Sect. 6.
3. The next lower priorities are assigned to the nodes that directly or indirectly invoke the modified or newly introduced nodes. A higher priority among these nodes is assigned to a node that is closer (in terms of path length) to a newly introduced or modified node.
4. All other nodes (except the nodes that are invoked by the modified or newly introduced nodes) are assigned the next lower priority in an arbitrary order.
5. The nodes that are invoked either directly or indirectly by the modified or newly introduced nodes are assigned the least priorities. These nodes may not even be tested in a regression testing scenario. Hence these nodes are not part of the *selected subgraph* for testing.

5.2 Prioritization of test cases for the ICGs

This prioritization deals with the inner level prioritization of test cases for the ICGs for each of the nodes in the FDG

within our two-level prioritization approach. As explained before, the inner prioritization of test cases is based on the ICG. We show the modified functional sub-modules for an ICG of a node in the FDG for the *Online Bookstore* application in Figs. 3 and 4. The prioritization strategy of the test cases for each ICG of the selected nodes of the FDG is same.

Assume that two new or modified functional sub-modules (nodes in the ICG) e_p and e_q for a node $f_k \in FDG$ have the associated test cases t_{kp} and t_{kq} . The *degree of modification* of a node in the ICG is determined by the changes in the lines of code. If node $e_p \in ICG$ has more changes than another node e_q , we say that e_p has a higher degree of modification. The prioritization scheme for the sub-modules in an ICG is as follows:

- A new node in an ICG is assigned the highest priority. If there are multiple sub-modules, they are assigned the highest priorities in an arbitrary order.
- The next lower priorities are assigned to the nodes in decreasing order of degree of modifications. If two nodes have the same degree of modification, the one closer to the root of the ICG is assigned a higher priority.
- The next lower priorities are assigned to the nodes that directly or indirectly invoke the new or modified nodes, with higher priorities assigned to nodes that are closer to the new or modified nodes.
- All other nodes are assigned the least priorities in an arbitrary order.

6 Prioritization strategies

Rothermel et al. (2001) defined the problem of test suite prioritization. Given T as a test suite, P is the set of all test suites that are the prioritized orderings of T obtained by permuting the tests of T and F is a function obtained from P to the reals, the problem is to find a permutation, $T' \in P$ such that $(\forall T'')(T'' \in P)[f(T') \geq f(T'')]$.

Prioritization can be based on any criteria (fault detection, code coverage and others (Do et al. 2006; Elbaum et al. 2000; Harrold et al. 2001; Rothermel et al. 2001; Sampath et al. 2008; Srivastava and Thiagarajan 2002)). In this section, we introduce different prioritization strategies for prioritizing test sets that contain test cases for specific functionalities in a web application. These functionalities are the nodes in the FDG. The dashed path (Fig. 5) shows the longest route from the root.

The suggested strategies are based on the possible routes in the FDG to detect faults early.

- *Position from root (longest route) high–low* This will identify the longest route of the modified nodes from

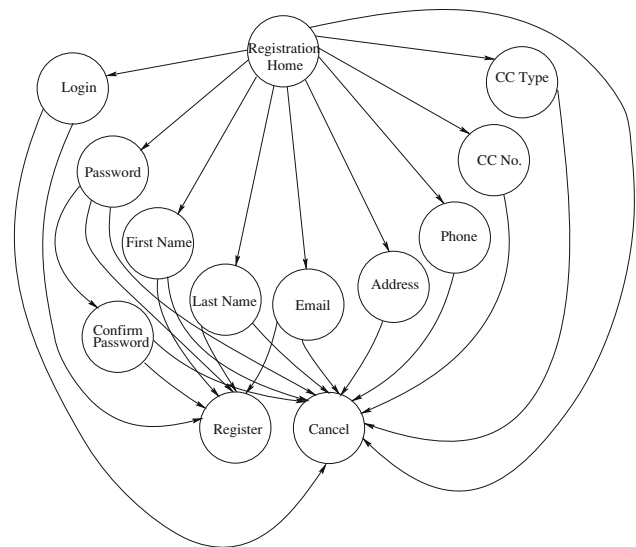


Fig. 3 Inter-procedural control flow graph (ICG) for the *Registration* node in the FDG of the *Online Bookstore* application

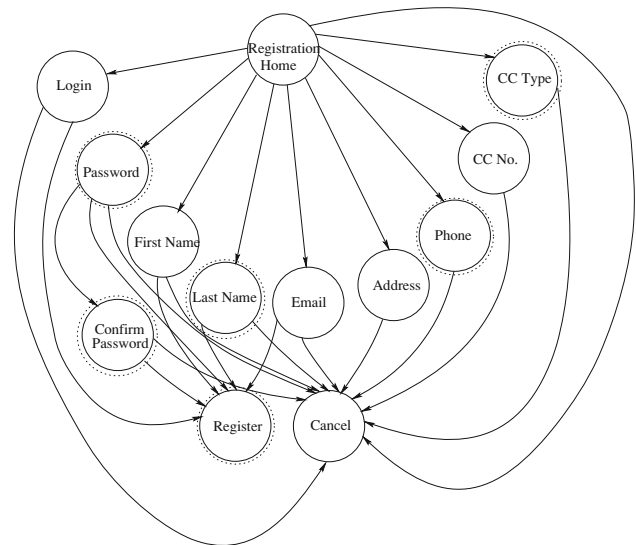


Fig. 4 Control graph-registration (modified functional sub-module)

the root in the FDG. The test cases related to nodes that have the longest routes are executed first.

- *Position from root (longest route) low–high* This will identify the longest route of the modified nodes from the root in the FDG. The test cases related to nodes that have the longest routes are executed last.
- *Position from root (shortest route) high–low* This will identify the shortest route of the modified nodes from the root in the FDG. The test cases related to nodes that have the shortest routes are executed first.
- *Position from root (shortest route) low–high* This will identify the shortest route of the modified nodes from the root in the FDG. The test cases related to nodes that have the shortest routes are executed last.

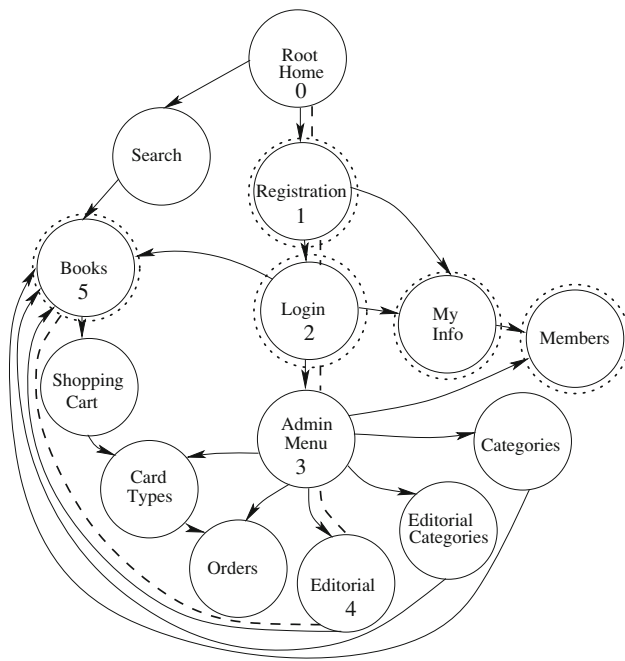


Fig. 5 Faults detected

- *No. of invoking functionalities (Max–Min)* The test cases related to the modified nodes in the FDG that invoke more functional modules are executed first.
- *No. of invoking functionalities (Min–Max)* The test cases related to the modified nodes that invoke more functional modules are executed last.
- *No. of changes (Max–Min)* The changes are identified as per the ICGs of the nodes of the FDG. The test cases related to nodes that have more changes/modifications are executed first.
- *No. of changes (Min–Max)* The changes are identified as per the ICGs of the nodes of the FDG. The test cases related to nodes that have more changes/modifications are executed last.
- *Random* Randomly permute the order of tests.

7 Experimental evaluation

In this section, we detect the different number of faults with various prioritization strategies and evaluate which prioritization strategy detects faults early in test suite execution. We selected an application *Online Bookstore* for our experiments. The *Online Bookstore* application is an online shopping portal for buying books. This application uses ASP for its frontend and MySQL for its backend connectivity. The application allows the users to search for the books by different keywords, add to the shopping cart and proceed to the orders. We seeded faults in the *Online Bookstore* application.

Experimental methodology We used C# to implement this approach. We generated 130 test cases of the *Online Bookstore* application and converted them to C# test scripts readable by the selenium test tool for automatic test suite execution. We categorized these test cases into different functionalities using the FDG of the web application and sliced the entire test suite into test sets. Each test set in a test suite is composed of test cases related to that specific functionality. We generated the test cases by following the activities of each module. The test cases were converted into a file format supported by the various automatic test case execution tools like selenium (Torsel 2011). The generated test cases were assumed to be non-redundant and generated as per the functional specifications.

We prioritized the test sets using our prioritization strategies and the test cases within the test set were prioritized using the ICGs. After prioritizing the test cases related to new/modified functionalities, we divided the entire test suite execution into two parts, one part deals with the test cases related to new/modified functionalities and these test cases are prioritized using our two-level prioritization approach and executed as a front-end activity in test suite execution. The remaining test cases are executed as a backend test suite execution.

Evaluation metrics Rothermel et al. (1996) presented a metric for measuring fault detection rates of test suites in a given order. This metric is called average percentage of faults detected (APFD). APFD values range from 0 to 1; higher numbers imply faster (better) fault detection rates (Elbaum et al. 2001). We have used a variation of the existing APFD metric for measuring fault detection rate. This form of the APFD metric helps us to calculate the results for incremental test suite execution.

The original metric used by Rothermel et al. (2001) is as follows:

$$APFD = 1 - \frac{TF_1 + TF_2 + \dots + TF_m}{mn} + \frac{1}{2n}$$

We have rewritten this metric in the following form for the incremental computation of APFD:

$$APFD = \frac{(n - TF_1) + (n - TF_2) + \dots + (n - TF_m)}{mn} + \frac{1}{2n}$$

TF_i is the position of first test in T that exposes fault i ;
 n = no. of test cases;
 m = no. of faults.

Informally, APFD measures the area under the curve that is plotted by the percentage of faults detected by prioritized test case order and the test suite fraction. To perform the experimental evaluation, we randomly seeded 20 faults in various modified functionalities of the *Online*

Table 1 Test case prioritization strategies

Modified functionality	Position from root (longest route) high–low	Position from root (longest route) low–high	Position from root (shortest route) high–low	Position from root (shortest route) low–high	No. of calling functionalities (max–min)	No. of calling functionalities (min–max)	No. of changes (max–min)	No. of changes (min–max)	Random
Reg	1	1	1	1	11	11	7	7	1
Login	2	2	2	2	10	10	1	1	2
MyInfo	3	3	2	2	1	1	2	2	3
Books	5	5	2	2	3	3	8	8	4
Members	4	4	3	3	0	0	2	2	5
Prioritized	–B	–R	–M	–R	–R	–M	–R	–MI	–R
Sequence	–M	–L	–B	–L	–L	–MI	–B	–L	–L
	–MI	–MI	–MI	–MI	–B	–B	–M	–M	–MI
	–L	–M	–L	–B	–MI	–L	–L	–B	–B
	–R	–B	–R	–M	–M	–R	–MI	–R	–M

Table 2 Results from prioritization strategies

% Age of test suite run	Position from root (longest route) high–low	Position from root (longest route) low–high	Position from root (shortest route) high–low	Position from root (shortest route) low–high	No. of invoking functionalities (max–min)	No. of invoking functionalities (min–max)	No. of changes (max–min)	No. of changes (min–max)	Random
10	49.84	58.57	58.57	90.45	77.14	67.57	78.07	66.68	36.49
20	91.95	94.11	91.88	94.49	93.49	92.26	94.49	91.38	40.76
30	91.95	94.11	91.88	94.49	93.49	92.26	94.49	91.38	40.76
40	91.95	94.11	91.88	94.49	93.49	92.26	94.49	91.38	40.76
50	91.95	94.11	91.88	94.49	93.49	92.26	94.49	91.38	46.30
60	91.95	94.11	91.88	94.49	93.49	92.26	94.49	91.38	46.30
70	91.95	94.11	91.88	94.49	93.49	92.26	94.49	91.38	55.64
80	91.95	94.11	91.88	94.49	93.49	92.26	94.49	91.38	56.95
90	91.95	94.11	91.88	94.49	93.49	92.26	94.49	91.38	56.95
100	91.95	94.11	91.88	94.49	93.49	92.26	94.49	91.38	57.03

Bold indicates the highest APFD results obtained from this prioritization strategy

Bookstore application. Three different kinds of faults were seeded in the application:

- Logical faults
- Form faults
- Appearance faults

We seeded faults in the various modified functionalities like *Registration*, *Members*, *MyInfo*, *Login* and *Books* and assumed that the faults behave like real faults.

Threats to validity We manually seeded the faults in the web application. The faults may not be evenly distributed among the functionalities. Although they are considered to be faults of equal severity, faults with different severity levels may vary the results. The test execution may differ depending on hardware. The functional test case execution time may differ due to the varying lengths of test cases.

8 Results and analysis

Table 1 shows the prioritized sequences of test sets using various prioritization strategies. In Table 1, *B* refers to test cases related to functional module *Books*, *M* refers to test cases related to functional module *Members*, *MI* refers to test cases related to functional module *My Info*, *L* refers to test cases related to functional module *Login* and *R* refers to test cases related to functional module *Registration*.

Table 2 shows the results obtained using different prioritization strategies by prioritizing test cases using the FDG and ICGs. The table shows the APFD results for the corresponding percentage of test suite run. Table 2 shows results in 10 % increments. The suggested prioritization strategies deliver higher APFD results in the first 10 % of test suite execution. *Position from root (longest route) high–low*

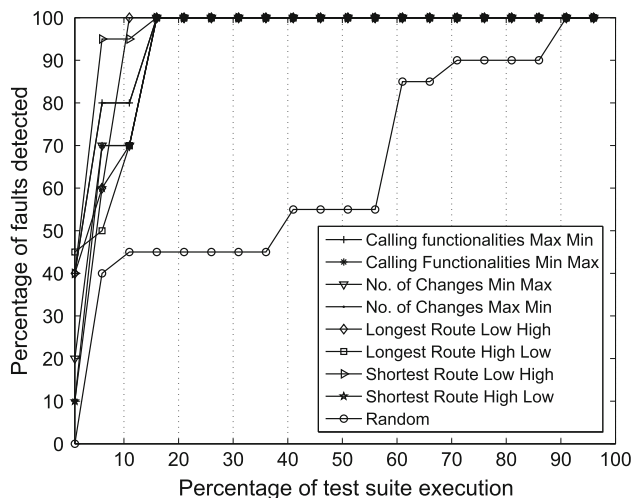


Fig. 6 Faults detected using various prioritization strategies

shows the lowest APFD results in the first 10 % of test suite execution. The experimental results indicate that *Position from route (shortest route) low–high* detects the maximum number of faults in the first 10 % of test suite execution. This is not surprising, as this strategy detects faults in the functional modules that are closer to the root of the FDG and hence this is a good strategy for detecting faults early.

Figure 6 shows the faults detected using various prioritization strategies. Random ordering of test suite execution shows the lowest APFD result as compared to all our prioritization strategies. Random ordering shows the APFD result of 57.03 even after 100 % of test suite execution. Our prioritization strategies are able to detect close to 100 % faults in the first 20 % of test suite execution. *No. of changes (max–min)* and *Position from route (shortest route) low–high* show the highest APFD of 94.49.

9 Conclusions and future work

We conclude that modified functionalities have more chances of faults. The already tested functionalities in the previous regression test cycles are locked in as per specifications. If the new/modified functionalities are receiving the correct values and parameters from the already tested functionalities, then we may not need to test those functionalities again. We suggested this approach using FDG and ICG graphs as the different modules are written using different languages in web applications. ICG graphs are constructed using the particular language used for that module. We suggested nine different prioritization strategies for prioritizing functional test cases. Of these prioritization strategies, if we prioritize the test cases based on the number of changes in different components of web applications, the components that have the maximum

number of changes/modifications have more chances of faults. But on the other hand, if we follow the shortest paths from the root for the modified functionalities and execute the tests accordingly, they are able to detect most of the faults in the first 10 % of test suite execution.

We validated the results using various different test combinations. In the future, we will validate our results on two more web applications. We will consider real faults with different cost levels. As we have shown, our first 20 % test suite execution detects most of the faults. In the future, we will suggest a technique that will select the test cases related only to the modified functionalities in web applications and execute only those test cases that will provide maximum fault detection. This may help to reduce the total test execution time as we need to execute only a subset of test cases.

References

- Baresi L, Pezzè M (2006) An introduction to software testing. *Electron Notes Theor Comput Sci* 148:89–111. doi:<http://dx.doi.org/10.1016/j.entcs.2005.12.014>
- Cartaxo E, Neto F, Machado P (2007) Test case generation by means of uml sequence diagrams and labeled transition systems. In: *Proceedings of the IEEE international conference on systems, man and cybernetics, ISIC*, pp 1292–1297. doi:[10.1109/ICSMC.2007.4414060](http://dx.doi.org/10.1109/ICSMC.2007.4414060)
- Di Lucca G, Fasolino A, Faralli F, De Carlini U (2002) Testing web applications. In: *Proceedings of the international conference on software maintenance*, pp 310–319. doi:[10.1109/ICSM.2002.1167787](http://dx.doi.org/10.1109/ICSM.2002.1167787)
- Di Lucca G, Fasolino A, Tramontana P, De Carlini U (2003) Abstracting business level uml diagrams from web applications. In: *Proceedings of the fifth IEEE international workshop on web site evolution*, pp 12–19. doi:[10.1109/WSE.2003.1234004](http://dx.doi.org/10.1109/WSE.2003.1234004)
- Do H, Rothermel G, Kinneer A (2006) Prioritizing junit test cases: an empirical assessment and cost-benefits analysis. In: *Empirical software engineering*, vol 11. Kluwer Academic Publishers, Hingham, pp 33–70. doi:[10.1007/s10664-006-5965-8](http://dx.doi.org/10.1007/s10664-006-5965-8)
- Elbaum S, Malishevsky AG, Rothermel G (2000) Prioritizing test cases for regression testing. In: *Proceedings of the 2000 ACM SIGSOFT international symposium on software testing and analysis, ISSTA '00*. ACM, New York, pp 102–112. doi:<http://doi.acm.org/10.1145/347324.348910>
- Elbaum S, Malishevsky A, Rothermel G (2001) Incorporating varying test costs and fault severities into test case prioritization. In: *Proceedings of the 23rd international conference on software engineering. IEEE Computer Society, ICSE '01*, Washington, DC, pp 329–338. doi:[10.1109/ICSE.2001.919106](http://dx.doi.org/10.1109/ICSE.2001.919106)
- Elbaum S, Malishevsky A, Rothermel G (2002) Test case prioritization: a family of empirical studies. In: *IEEE transactions on software engineering*, vol 28. IEEE Press, Piscataway, pp 159 – 182. doi:[10.1109/32.988497](http://dx.doi.org/10.1109/32.988497)
- Elbaum S, Karre S, Rothermel G (2003) Improving web application testing with user session data. In: *Proceedings of the 25th international conference on software engineering*, pp 49 – 59. doi:[10.1109/ICSE.2003.1201187](http://dx.doi.org/10.1109/ICSE.2003.1201187)
- Elbaum S, Rothermel G, Kanduri S, Malishevsky AG (2004) Selecting a cost-effective test case prioritization technique. In:

- Software quality control, vol 12. Kluwer Academic Publishers, Hingham, pp 185–210. doi:[10.1023/B:SQJO.0000034708.84524.22](https://doi.org/10.1023/B:SQJO.0000034708.84524.22)
- Garg D, Datta A (2012) Test case prioritization due to database changes in web applications. In: Proceedings of the 2012 IEEE fifth international conference on software testing, verification and validation, ICST '12. IEEE Computer Society, Washington, DC, pp 726–730. doi:[10.1109/ICST.2012.163](https://doi.org/10.1109/ICST.2012.163)
- Gonzalez-Sanchez A, Piel r, Abreu R, Gross HG, van Gemund AJC (2011) Prioritizing tests for software fault diagnosis. In: Software: practice and experience. Wiley, New York, pp 42–51. doi:[10.1002/spe.1065](https://doi.org/10.1002/spe.1065)
- Guo Y, Sampath S (2008) Web application fault classification—an exploratory study. In: Proceedings of the second ACM-IEEE international symposium on empirical software engineering and measurement, ESEM '08. ACM, New York, pp 303–305. doi:<http://doi.acm.org/10.1145/1414004.1414060>
- Harrold MJ, Jones JA, Li T, Liang D, Orso A, Pennings M, Sinha S, Spoon SA, Gujarathi A (2001) Regression test selection for Java software. SIGPLAN Not 36:312–326. doi: <http://doi.acm.org/10.1145/504311.504305>
- Heinecke A, Bruckmann T, Griebel T, Gruhn V (2010) Generating test plans for acceptance tests from uml activity diagrams. In: Proceedings of the 17th IEEE international conference and workshops on engineering of computer based systems (ECBS), pp 57–66. doi:[10.1109/ECBS.2010.14](https://doi.org/10.1109/ECBS.2010.14)
- Jeffrey D, Gupta N (2006) Test case prioritization using relevant slices. In: Proceedings of the 30th annual international computer software and applications conference, COMPSAC '06, vol 1, pp 411–420. doi:[10.1109/COMPSAC.2006.80](https://doi.org/10.1109/COMPSAC.2006.80)
- Li G, Lu K, Zhang Y, Lu X, Zhang W (2009) Decoupling binary-level dynamic test generation from specific architecture details. In: Proceedings of the fourth international conference on computer sciences and convergence information technology, ICCIT '09, pp 1041–1046. doi:[10.1109/ICCIT.2009.118](https://doi.org/10.1109/ICCIT.2009.118)
- Lucca GAD, Fasolino AR (2006) Testing web-based applications: The state of the art and future trends. In: Quality assurance and testing of web-based applications, vol 48, pp 1172–1186. doi:[10.1016/j.infsof.2006.06.006](https://doi.org/10.1016/j.infsof.2006.06.006)
- Mansour N, Houri M (2006) Testing web applications. In: Information and software technology, vol 48, pp 31–42. doi:[10.1016/j.infsof.2005.02.007](https://doi.org/10.1016/j.infsof.2005.02.007)
- Rothermel G, Harrold MJ (1996) Analyzing regression test selection techniques. In: IEEE transactions on software engineering, vol 22. IEEE Press, Piscataway, pp 529–551. doi:<http://dx.doi.org/10.1109/32.536955>
- Rothermel G, Harrold MJ, Dedhia J (2000) Regression test selection for C++ software. Softw Test Verif Reliab 10(2):77–109. doi:[10.1002/1099-1689\(200006\)10:2<77::AID-STVR197>3.0.CO;2-E](https://doi.org/10.1002/1099-1689(200006)10:2<77::AID-STVR197>3.0.CO;2-E)
- Rothermel G, Untch R, Chu C, Harrold M (2001) Prioritizing test cases for regression testing. IEEE Trans Softw Eng 27(10): 929–948. doi:[10.1109/32.962562](https://doi.org/10.1109/32.962562)
- Salehie M, Li S, Tahvildari L, Dara R, Li S, Moore M (2011) Prioritizing requirements-based regression test cases: a goal-driven practice. In: Proceedings of the 15th european conference on software maintenance and reengineering (CSMR), pp 329–332. doi:[10.1109/CSMR.2011.46](https://doi.org/10.1109/CSMR.2011.46)
- Sampath S, Sprengle S, Gibson E, Pollock L, Greenwald A (2007) Applying concept analysis to user-session-based testing of web applications. IEEE Trans Softw Eng 33(10):643–658. doi:[10.1109/TSE.2007.70723](https://doi.org/10.1109/TSE.2007.70723)
- Sampath S, Bryce R, Viswanath G, Kandimalla V, Koru A (2008) Prioritizing user-session-based test cases for web applications testing. In: Proceedings of the 1st international conference on software testing, verification, and validation, pp 141–150. doi:[10.1109/ICST.2008.42](https://doi.org/10.1109/ICST.2008.42)
- Samuel P, Mall R, Sahoo S (2005) Uml sequence diagram based testing using slicing. In: 2005 annual IEEE, INDICON, pp 176–178. doi:[10.1109/INDICON.2005.1590149](https://doi.org/10.1109/INDICON.2005.1590149)
- Srikanth H, Williams L, Osborne J (2005) System test case prioritization of new and regression test cases. In: ISESE, pp 64–73. doi:[10.1109/ISESE.2005.1541815](https://doi.org/10.1109/ISESE.2005.1541815)
- Srivastava A, Thiagarajan J (2002) Effectively prioritizing tests in development environment. In: Proceedings of the 2002 ACM SIGSOFT international symposium on software testing and analysis, ISSTA '02. ACM, New York, pp 97–106. doi:<http://doi.acm.org/10.1145/566172.566187>
- Torsell AM (2011) Automated test case generation for web applications from a domain specific model. In: Proceedings of the IEEE 35th annual conference on computer software and applications conference workshops (COMPSACW), pp 137–142. doi:[10.1109/COMPSACW.2011.32](https://doi.org/10.1109/COMPSACW.2011.32)
- Tung YH, Tseng SS, Lee TJ, Weng JF (2010) A novel approach to automatic test case generation for web applications. In: Proceedings of the 10th international conference on quality software (QSIC), pp 399–404. doi:[10.1109/QSIC.2010.33](https://doi.org/10.1109/QSIC.2010.33)
- Zimmermann T, Nagappan N (2007) Predicting subsystem failures using dependency graph complexities. In: Proceedings of the 18th IEEE international symposium on software reliability, ISSRE '07, pp 227–236. doi:[10.1109/ISSRE.2007.19](https://doi.org/10.1109/ISSRE.2007.19)